KLAi

Forests, boosting and overfitting: a stronger model without fooling ourselves

How do we get a stronger model without fooling ourselves?

AI and Data Science Professional | Session 17 of 24 | Module D: Machine learning
KLAi Professional Learning | Jotham Martin Wambi

Today in one view: what you will leave with

By the end of today you can



Explain overfitting, underfitting and the bias-variance trade-off from training error and test error

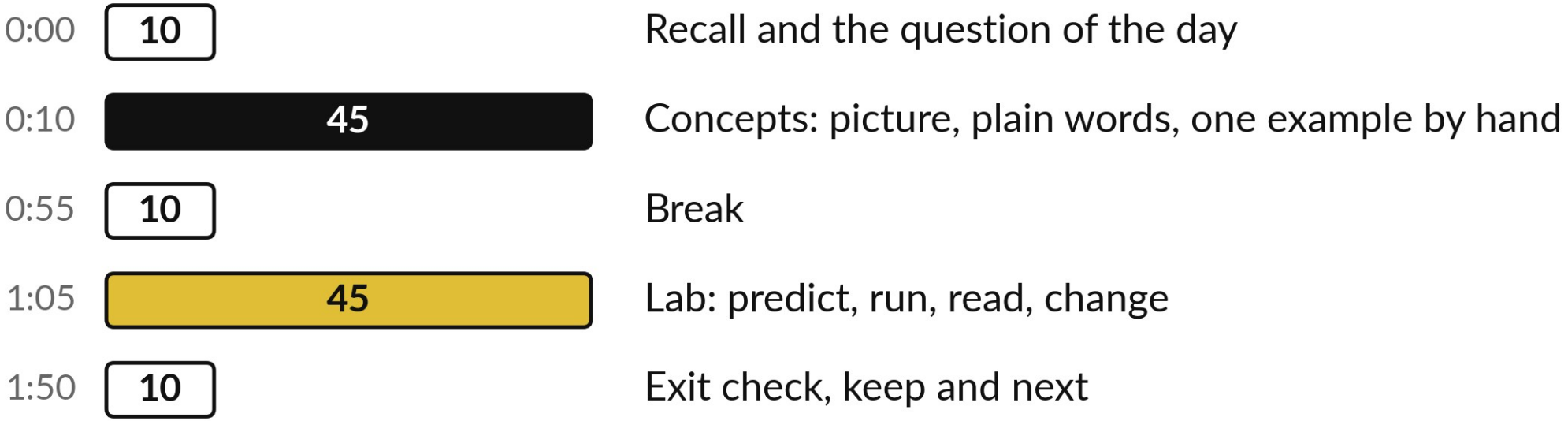


Describe how bagging (a random forest) and gradient boosting build an ensemble, and tune their hyperparameters with cross-validation



Read logistic regression coefficients and permutation importance without over-claiming, and ship the simpler model when scores are close

Your two hours: concepts first, then a Python lab



Ideas with pictures first. Then you open the notebook. Then the checks and what you keep.

Three questions from last time

1. A loan model scored far too well after one column was added. What kind of column does that, and what is it called?

2. Why use 5-fold cross-validation instead of one train and test split?

3. What does an AUC of 0.5 mean?



Answer aloud first. Then we check together.

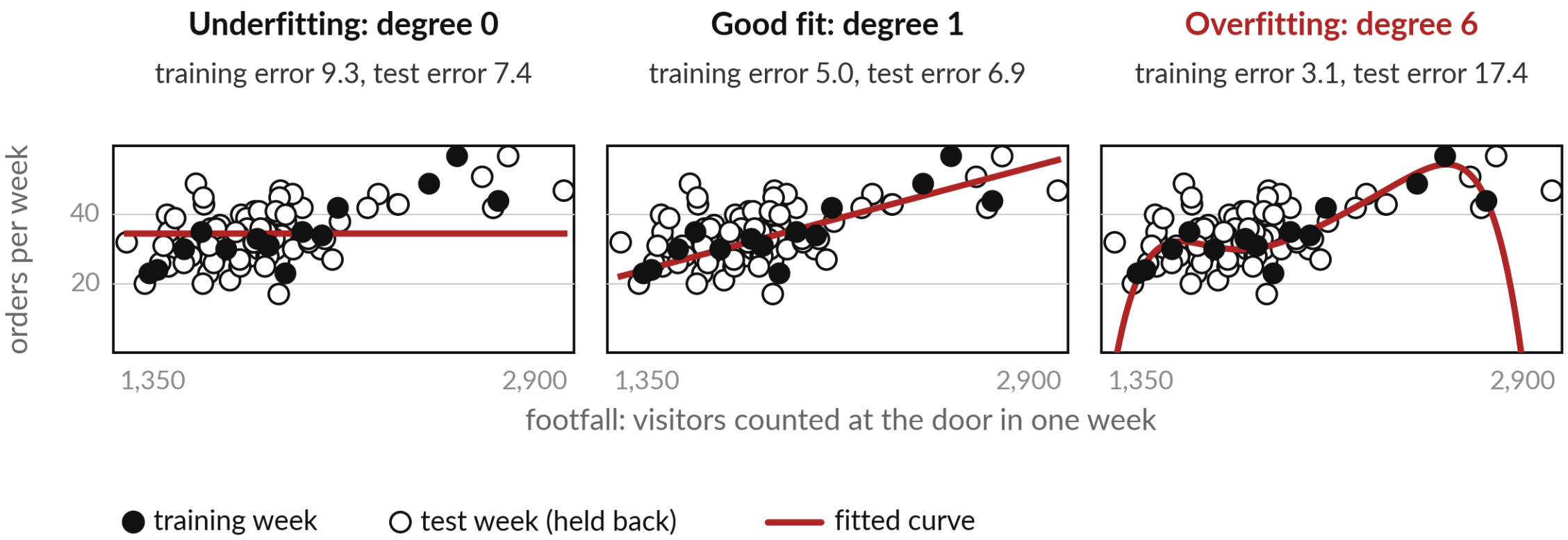
Answers to the three questions from Session 16

Question	Short answer
A loan model scored far too well after one column was added. What kind of column does that, and what is it called?	A column that is only recorded after the outcome is known, here collection_calls. It is data leakage.
Why use 5-fold cross-validation instead of one train and test split?	Each part of the data takes a turn as the test fold, so you get five scores and their average depends less on luck.
What does an AUC of 0.5 mean?	The model ranks a defaulted loan above a repaid one no better than a coin toss.

Words we use today

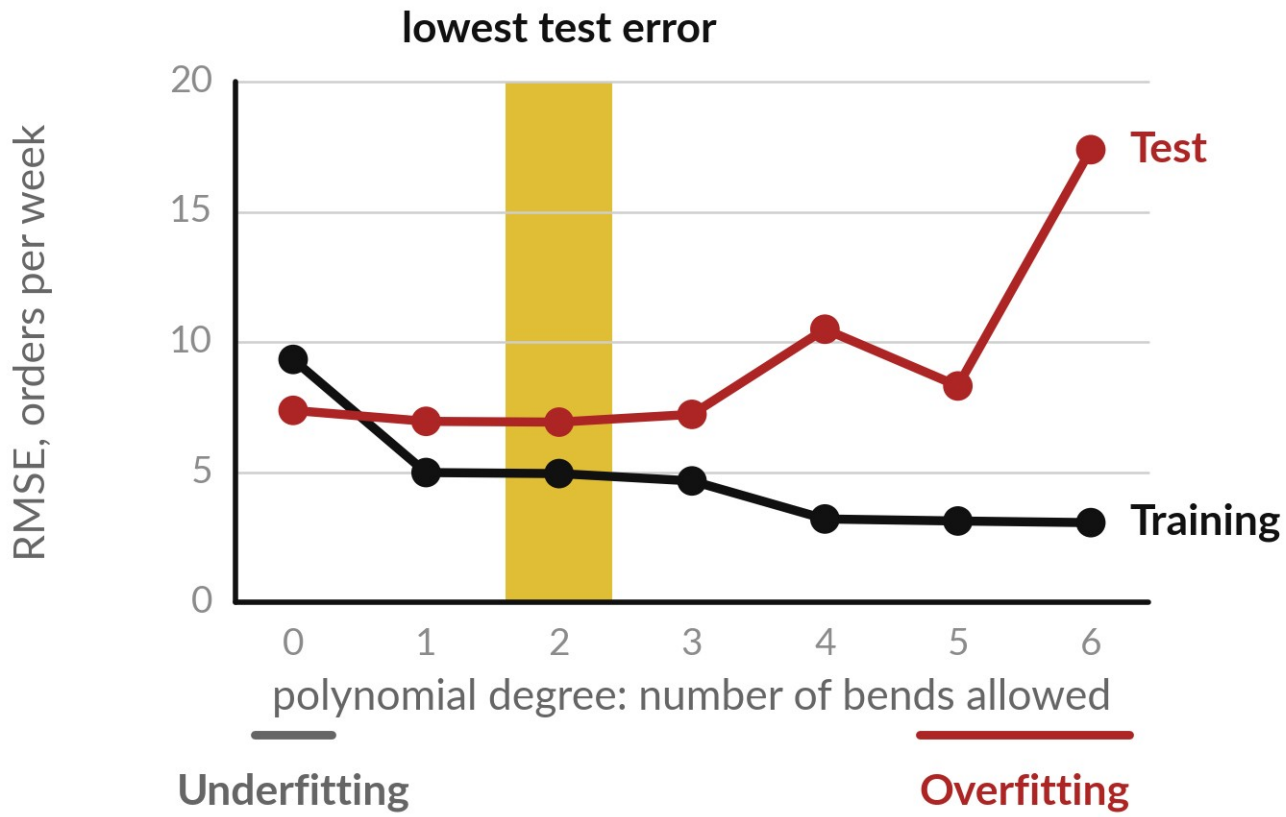
	Word	What it means
	**Overfitting and underfitting**	Overfitting: the model fits noise, so training rows score well and new rows score worse. Underfitting: too simple to see the pattern.
	**Ensemble (random forest, gradient boosting)**	Many small models combined into one. A forest averages trees (bagging). Boosting adds trees, each fitting the residual errors so far.
	**Hyperparameter**	A setting chosen before training, such as max_depth. Parameters are learned from the data. Tuning tries combinations with cross-validation.
	**Permutation importance**	The drop in test score when one variable is shuffled. It shows what the model relies on. It does not show a cause.
	**Parsimony (model selection)**	When scores are close, ship the simpler model. It is easier to explain, check and maintain.

Underfitting, a good fit and overfitting on 15 training weeks at the Jinja branch



Each dot is one week. Degree is the number of bends the red curve may have.

Training error keeps falling as the curve gets more flexible, but test error rises again



Training error always improves. Test error is the honest score.

Training error: 9.3 down to 3.1.

Test error: lowest at degrees 1 and 2 (6.9), then 17.4 at degree 6.

RMSE: the typical miss, in orders per week.

The bias-variance trade-off: two ways to be wrong

High bias: underfitting

Bias is error from a model too simple for the pattern.

Wrong on training and new rows alike.

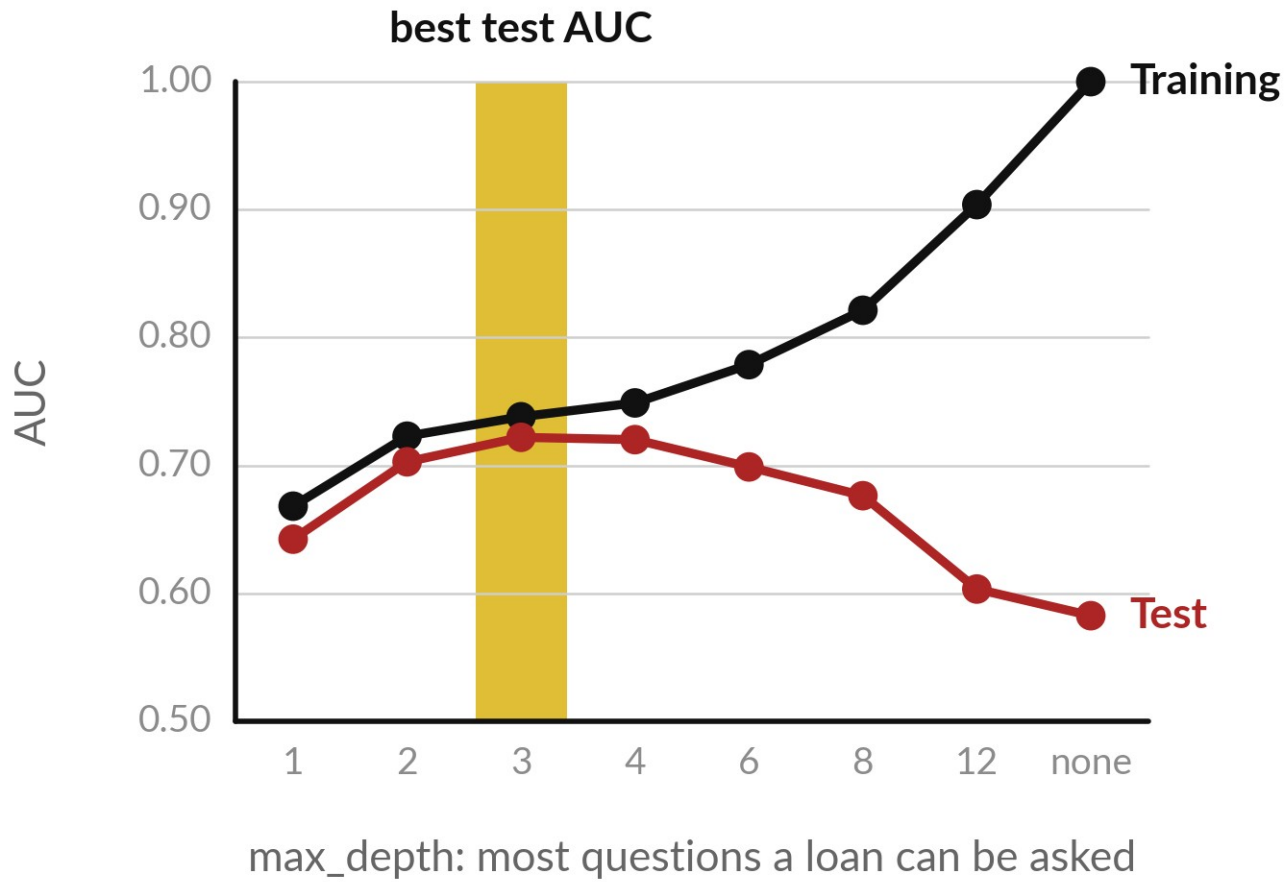
High variance: overfitting

Variance is how much the model changes when the training rows change.

Right on training rows, wrong on new rows.

More flexibility lowers bias and raises variance. Pick the best test score.

A decision tree overfits as max_depth grows: training AUC reaches 1.00 and test AUC falls



max_depth: the most yes or no questions a loan can be asked.

Depth 3: train 0.738, test 0.722.

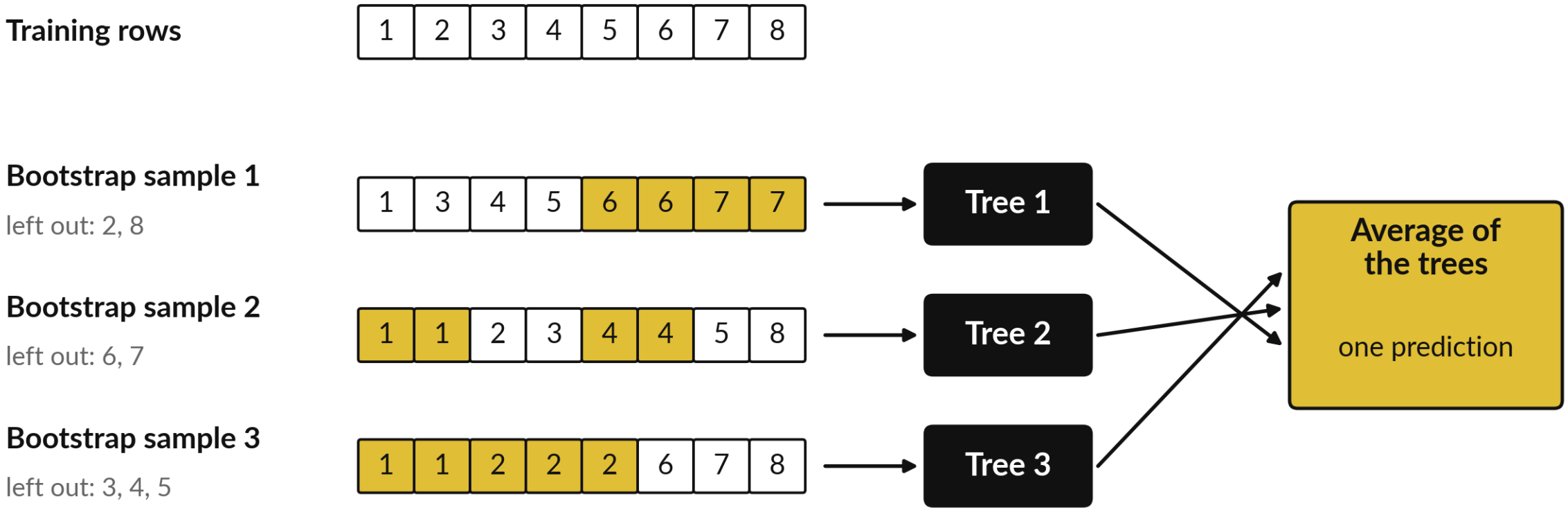
No limit (31 deep): train 1.000, test 0.583.

Gap = train AUC minus test AUC.

Parameters are learned from the data. Hyperparameters are chosen before training

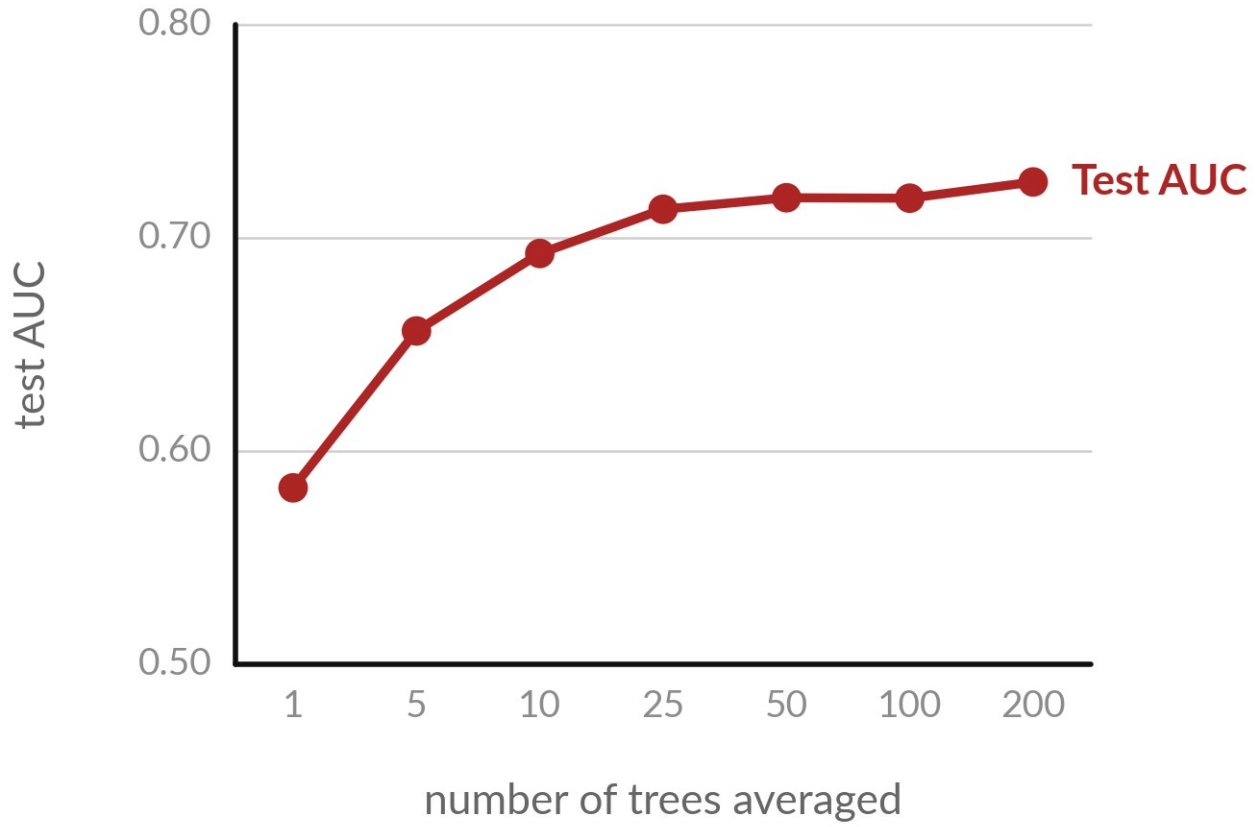
	Parameter	Hyperparameter
Who sets it	The learning algorithm, from the training rows	You, before training starts
In a decision tree	Each split question and its threshold	max_depth, min_samples_leaf
In a random forest or boosting	The splits inside every tree	n_estimators, max_features, learning_rate
In logistic regression	The coefficients and the intercept	C, the strength of the regularisation penalty (default 1.0, L2). The penalty makes the coefficients depend on the scaling
How to choose it	Fit the model	Hyperparameter tuning with cross-validation

Bagging: grow many trees on bootstrap samples and average their predictions



Gold rows were drawn twice or more (x2, x3). About 37 percent are left out. Illustrative: 8 rows.

Averaging many trees lifts test AUC from 0.58 for one tree to 0.72 for a forest



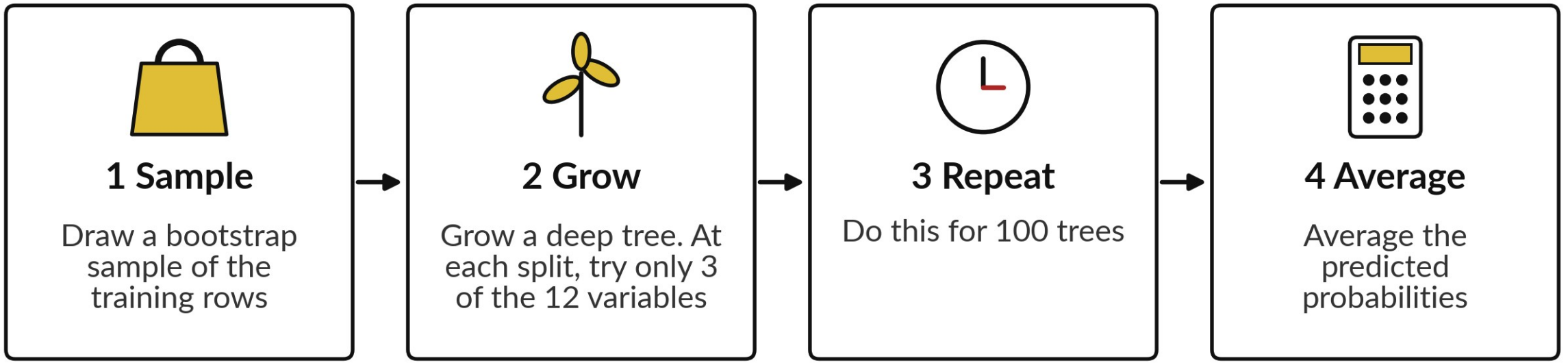
More trees help up to about 50. After that the curve is flat and small ups and downs are noise.

One tree: test AUC 0.576 on average (0.547 to 0.615).

Forest of 100 trees: 0.719.

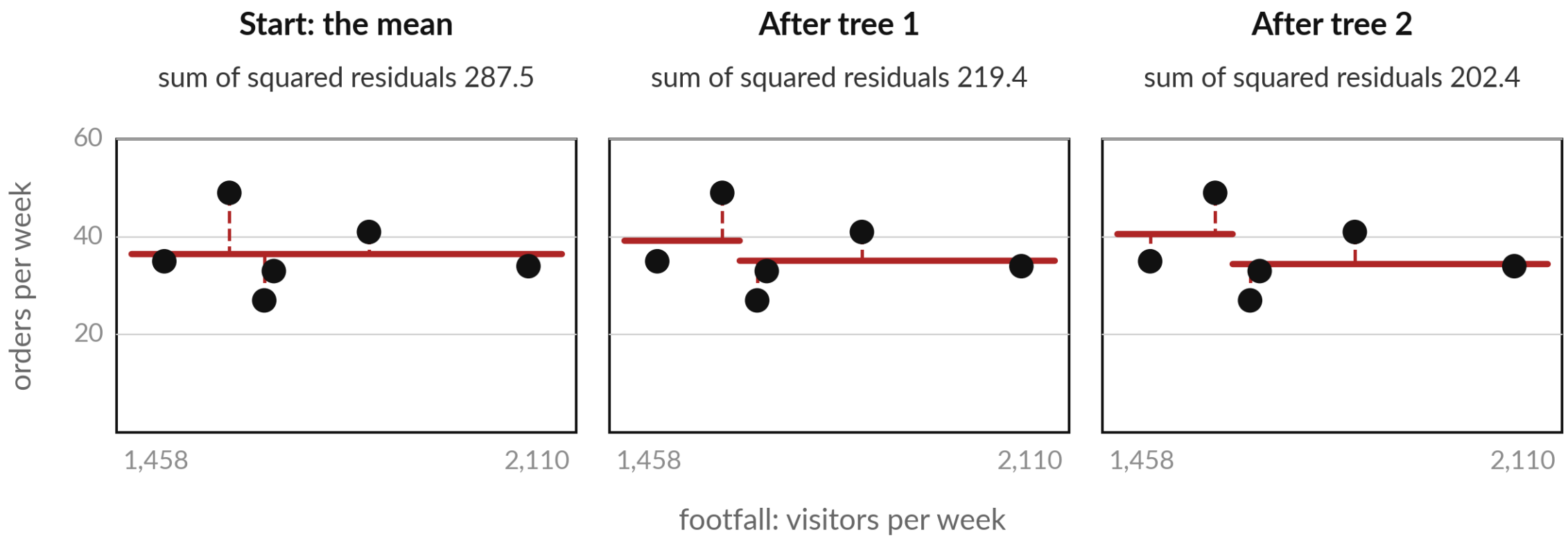
Training AUC is still 1.000.

A random forest is bagging plus a random subset of variables at every split



An ensemble is many models combined into one prediction. Test AUC on our loans: bagging with all 12 variables 0.709, random forest 0.719 (100 trees).

Gradient boosting: each new tree fits the residual errors of the ensemble so far



Red line: prediction. Dashed: residual (actual minus predicted). Learning rate 0.5.

The arithmetic of one boosting step on six weeks

Footfall	Orders	Start: mean	Residual	Tree 1 adds	Prediction now	New residual
1,518	35	36.5	-1.5	+2.750	39.250	-4.250
1,613	49	36.5	+12.5	+2.750	39.250	+9.750
1,664	27	36.5	-9.5	-1.375	35.125	-8.125
1,678	33	36.5	-3.5	-1.375	35.125	-2.125
1,817	41	36.5	+4.5	-1.375	35.125	+5.875
2,050	34	36.5	-2.5	-1.375	35.125	-1.125

Tree 1 asks: is footfall at most 1,638? The mean residual is +5.5 on the yes side and -2.75 on the no side. The learning rate 0.5 halves each correction.

Forest or boosting: two ways to build an ensemble

Random forest (bagging)

Deep trees grow side by side on bootstrap samples.

Each tree overfits. Averaging cancels the noise.

Settings: n_estimators, max_depth, max_features.

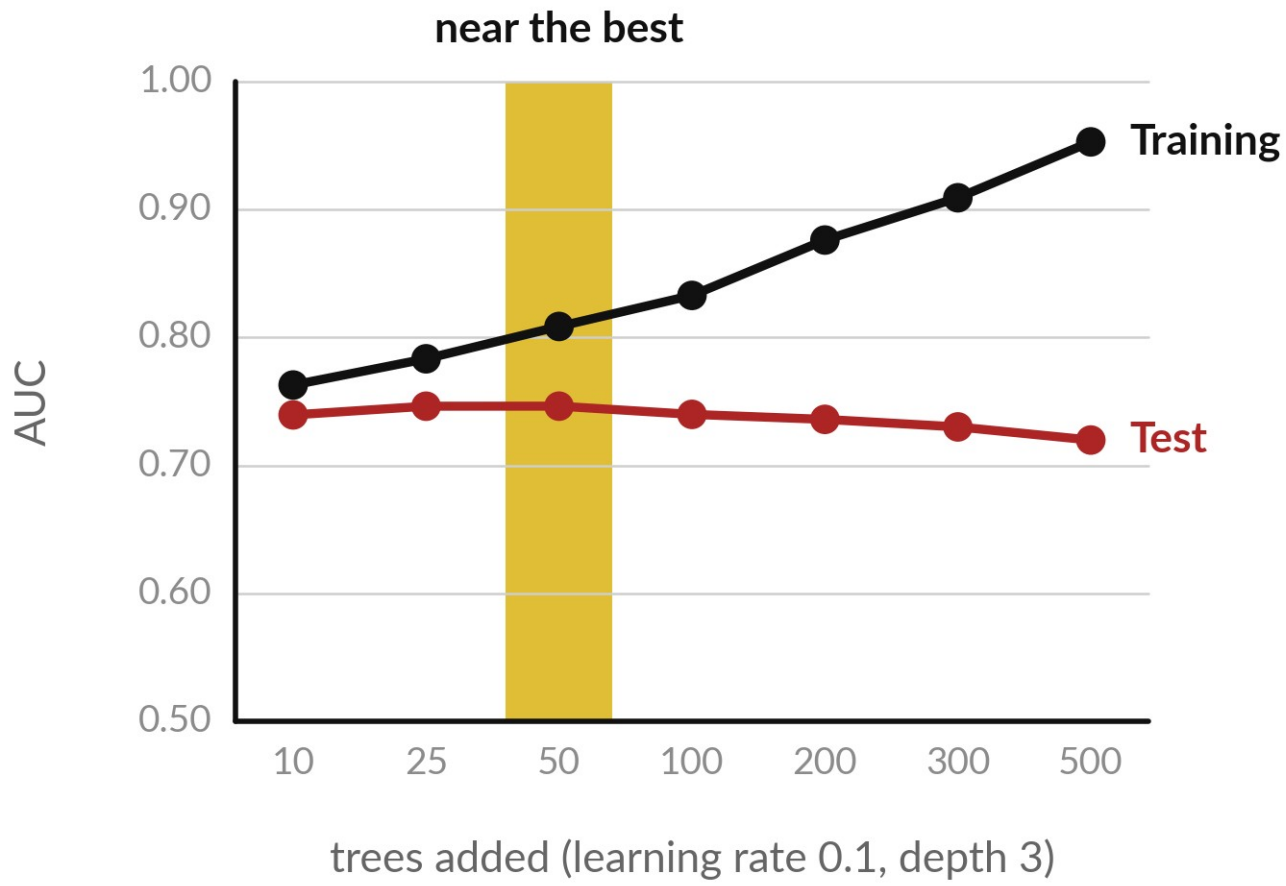
Gradient boosting

Shallow trees grow in a sequence, each fitting the residual errors so far.

Settings: n_estimators, learning_rate, max_depth.

Both are ensembles. Both need tuning, and both can overfit.

Boosting keeps improving the training AUC, but the test AUC peaks near 45 trees



Past the peak, extra trees fit noise.

50 trees: train 0.809, test 0.746.

500 trees: train 0.953, test 0.720.

Best test AUC 0.747, at 45 trees. Picked from the test curve, for illustration only.

Hyperparameter tuning: grid search scores every combination with 5-fold cross-validation

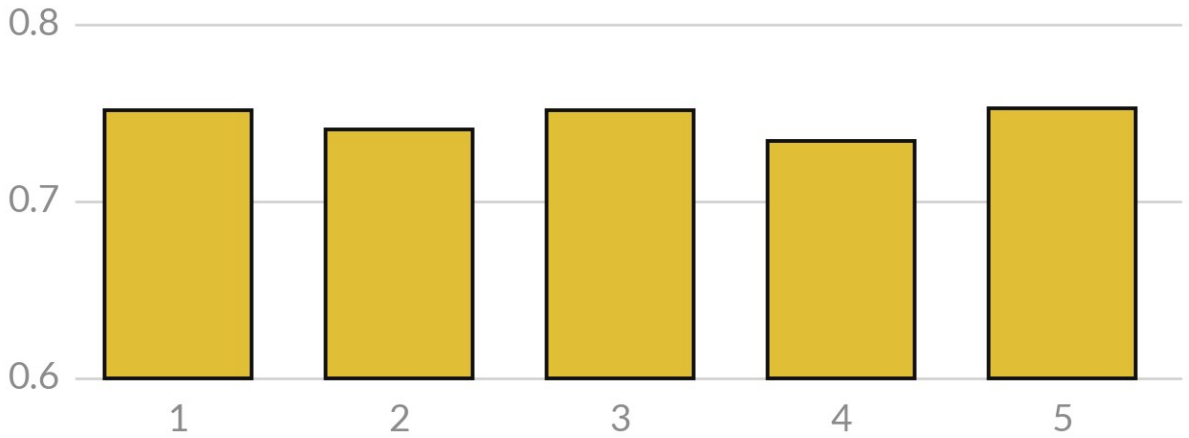
max_depth

min_samples_leaf

	1	10	30
3	0.739	0.740	0.738
6	0.743	0.745	0.744
none	0.722	0.736	**0.746**

Each cell: mean AUC over 5 folds

The five folds of the best cell



mean 0.746, sd 0.007

9 combinations × 5 folds = 45 fits. The best cell, refitted, scores 0.748 on the test rows.

Logistic regression models the log-odds of default as a weighted sum of the variables

Log-odds (logit)

$$z = b_0 + b_1 x_1 + b_2 x_2 + \ \ldots \ + b_{12} x_{12}$$

Chance of default (sigmoid)

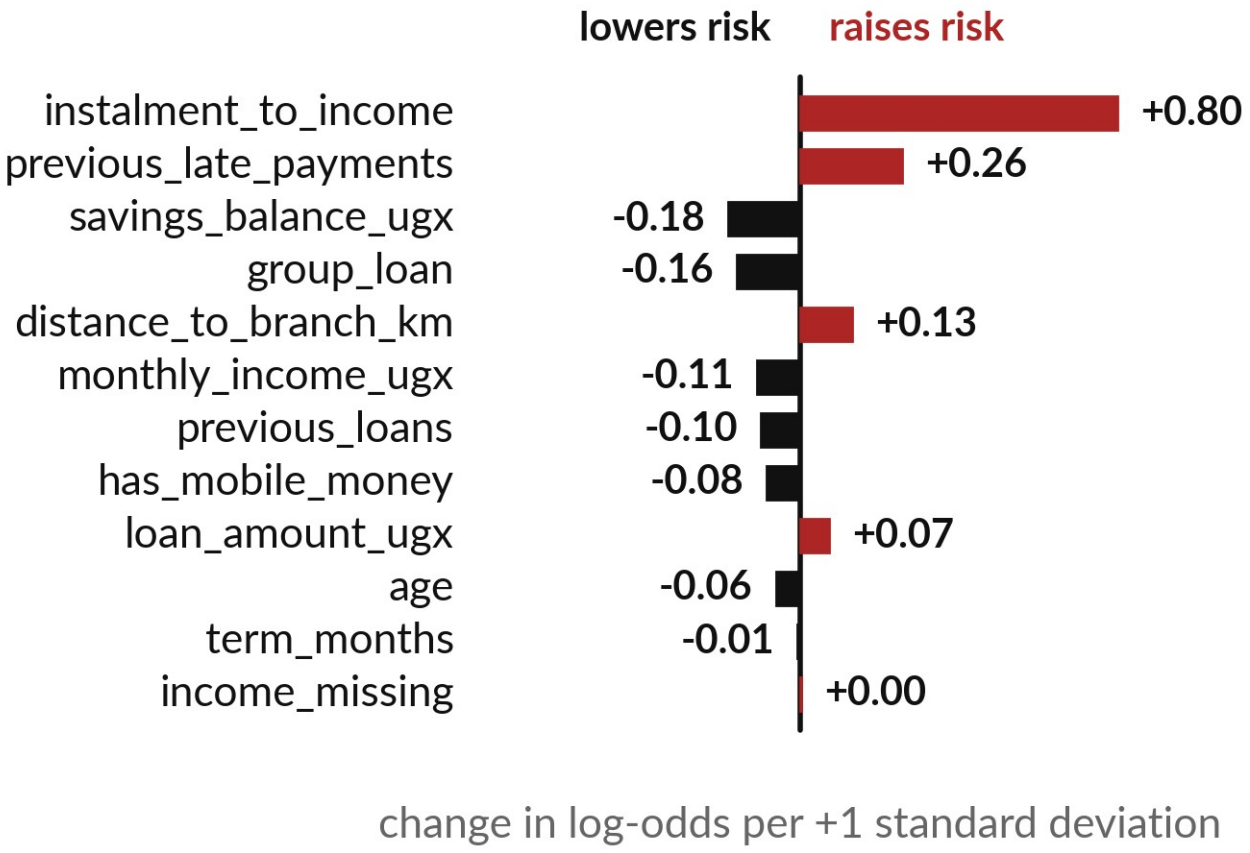
$$p = \frac{1}{1 + e^{-z}}$$

One test loan

$$z = -2.41 \quad p = 0.082$$

Symbol	What it means	In our data
b_0, b_1, ...	Intercept and coefficients, learned from the training rows	b for the ratio = +0.80
x_1, ..., x_12	The 12 variables, each scaled to mean 0 and standard deviation 1 (the 12 columns of Session 14)	age, income, ...
x_12 ratio	instalment_to_income: loan x 1.15 / term in months / monthly income. The 1.15 is an illustrative assumption	median 0.29
z and p	z: the log-odds, ln(p / (1 - p)). p: the chance of default	z -2.41, p 0.082

Standardised coefficients: the instalment-to-income ratio and previous late payments raise the log-odds of default



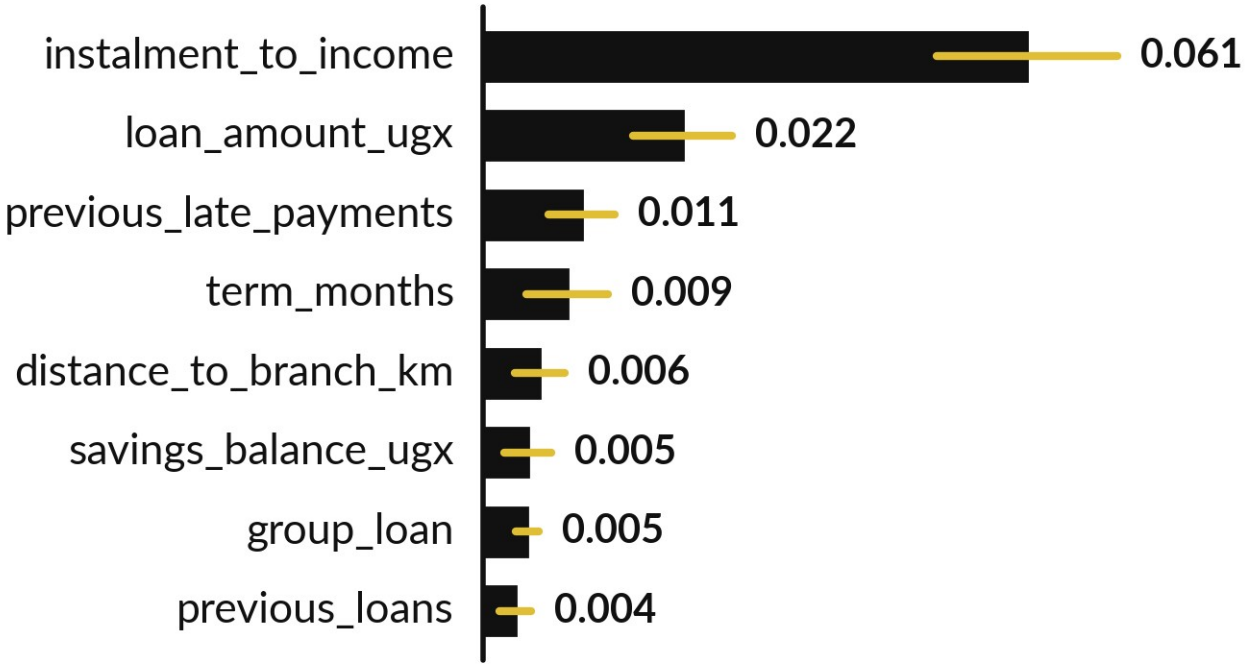
Sign gives the direction. Length gives the size.

instalment_to_income +0.80: odds of default x 2.2 per standard deviation.

late payments +0.26, group_loan -0.16.

Associations only.

Permutation importance: shuffle one variable and see how far the test AUC falls



drop in test AUC (gold line: spread over 10 repeats)

A variable the model relies on hurts the score when shuffled.

Shuffle one test column, rescore, repeat 10 times. Pass scoring="roc_auc", or the drop is in accuracy.

instalment_to_income: AUC falls 0.061. loan_amount_ugx: 0.022.

age: about zero.

Permutation importance has limits, and you must state them

What it tells you

Which variables this fitted model relies on, measured on test rows.

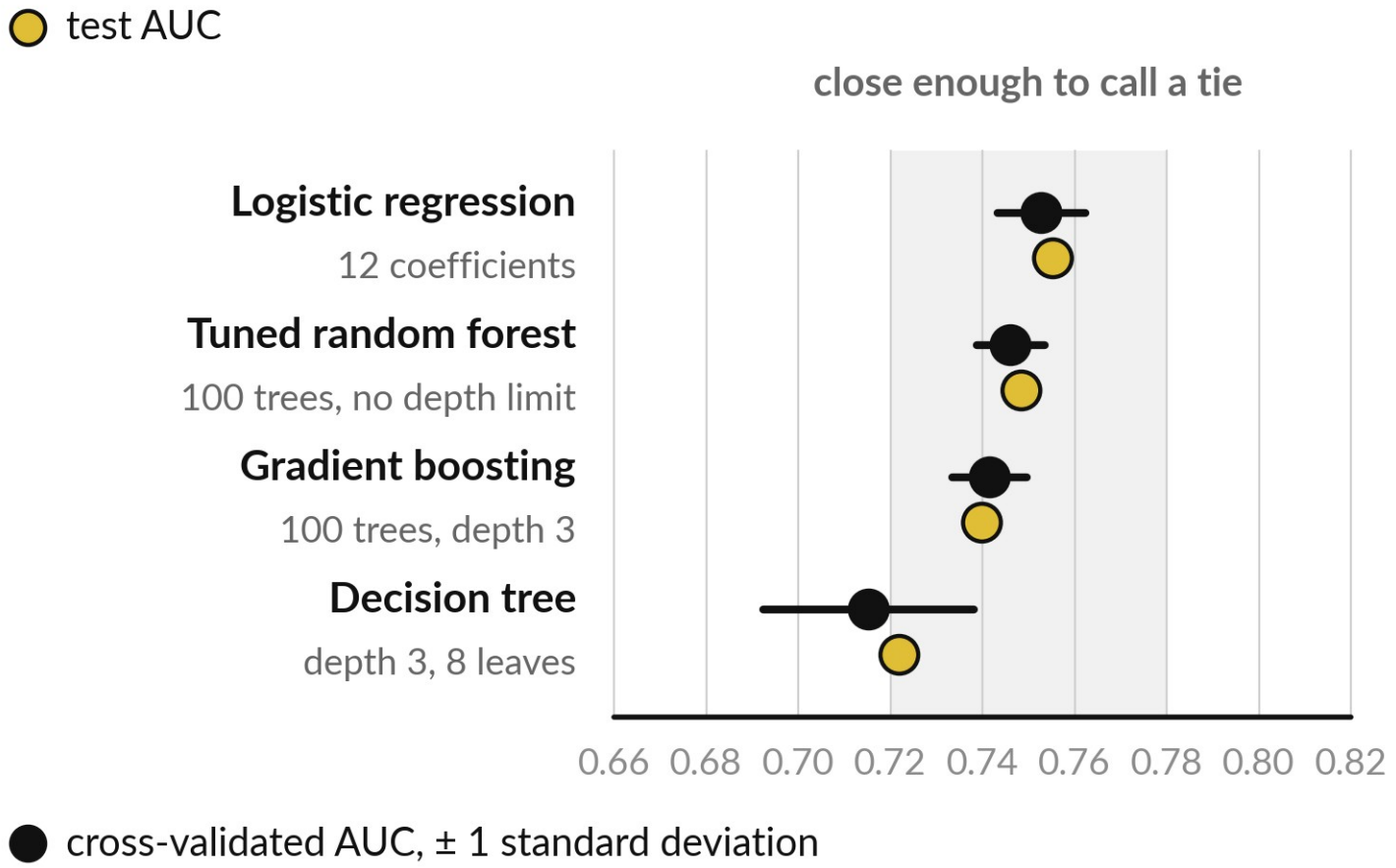
What it cannot tell you

Correlated variables share the credit. With a copy added, instalment_to_income fell from 0.061 to 0.021; the copy scored 0.030.

It shows no cause.

Say: this model relies on the ratio. Avoid saying the ratio causes default.

Model selection: the scores are close, so ship the simpler model (parsimony)



The top three scores differ by about 0.01, about as much as one fold differs from the next.

Cross-validated AUC: 0.753, 0.746, 0.742.

Test AUC: 0.755, 0.748, 0.740.

Parsimony: prefer the simpler model.

Decision: which model would you ship to a credit committee?

A credit committee asks which model to ship. What do you recommend, and what would change your mind?



Logistic regression

Test AUC 0.755. Twelve readable coefficients. Easy to explain to the committee.



Tuned random forest

Test AUC 0.748. Strong, but a hundred trees to explain and to monitor.



Gradient boosting

Test AUC 0.740. Needs the number of trees tuned with care.

Write your choice and one reason in your notes. We compare after the lab.

Where you will use this at work: compare models fairly before you recommend one

Your job	The model question you meet	What today lets you do
Credit analyst at a bank or SACCO, using scikit-learn	A vendor says a boosted model beats the scorecard.	Ask for cross-validated and test AUC, and the training-test gap, before you agree.
Data analyst on a mobile money fraud team	A forest flags thousands of transactions.	Tune the number of trees and depth with GridSearchCV, and keep logistic regression as the baseline.
Health records officer at a district office	Clinicians ask which factors drive a missed appointment.	Report coefficients and permutation importance, and say they show association only.
Team lead who reviews a model report	The report shows one training score.	Ask for the test score, the tuning method and a simple benchmark before the model is approved.

Read the recipe before the lab: fit, score twice, then tune

```
model = RandomForestClassifier(random_state=42)
model.fit(X_train, y_train)
p_test = model.predict_proba(X_test)[:, 1]
auc_test = roc_auc_score(y_test, p_test)
grid = {"max_depth": [3, 6, None]}
search = GridSearchCV(model, grid, cv=5,
                      scoring="roc_auc")
search.fit(X_train, y_train)
```

In plain words

Lines 1 and 2 fit a forest (100 trees by default).

Line 3: the predicted chance of default for each test loan.

Line 4 scores it. Score the training rows too, to see the gap.

The last lines try each max_depth with 5-fold cross-validation.

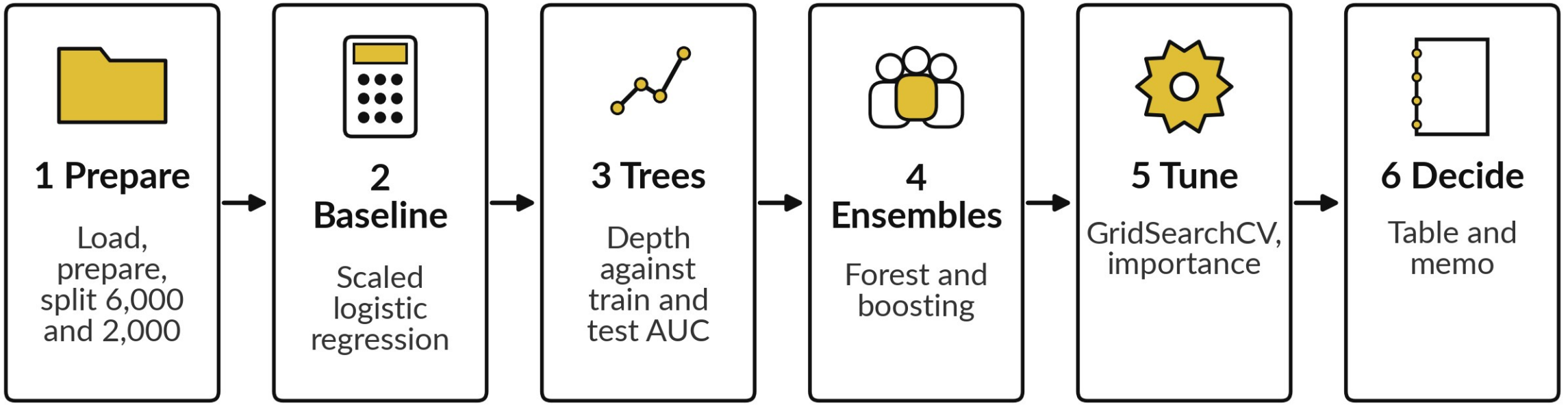
search.best_params_ holds the winner.



Part 2: the Python lab

Close the slides. Open the notebook.

The lab has six parts, and you keep a comparison table and a decision memo



Eight tasks in 45 minutes. In each task, predict first, run second, read what the output tells you, then change one thing.

Check your numbers before you move on

What you compute	You should get
Loans kept, defaults, default share	8,000, 1,004, 12.55 percent
Training rows and test rows (defaults in each)	6,000 (753) and 2,000 (251)
Logistic regression: train and test AUC	0.761 and 0.755
Tree: test AUC at depth 3. Train AUC, no limit	about 0.722. 1.000
Forest, 100 trees: train and test AUC	about 1.00 and 0.72 (within 0.03)
Boosting, 100 trees: train and test AUC	0.833 and 0.740
Grid search: best settings, mean AUC	4 cells tie within 0.005; mean about 0.746
Top permutation importance	instalment_to_income, about 0.061

The median income is computed on all 8,000 loans before the split, on purpose (simple, tiny leak). Session 20 shows the leak-free way. Forests use random numbers.

Five likely mistakes in the lab and how to fix them

	What you see	What it means	What to do
	**Forest training AUC about 1.00, test AUC about 0.72**	That is correct. The forest memorises its training rows.	Write it down as the generalisation gap. Leave it as it is.
	**Test AUC about 0.99**	collection_calls leaked back into X.	Build X from the 12 listed columns only.
	**ValueError: could not convert string to float**	X still holds text columns such as district.	Select the 12 numeric columns by name.
	**GridSearchCV runs for minutes**	It fits 45 forests.	Add n_jobs=-1 and run it once. Skip rerunning the cell.
	**Scores differ in the third decimal**	Random numbers and library versions.	Set random_state=42. Accept the ranges in the check table.

Exit check: three questions, then compare with the short answers

	Question	Short answer
	**A model scores AUC 1.00 on the training rows and 0.58 on the test rows. What is happening, and what is the gap called?**	Overfitting: it memorised the training rows. The difference is the generalisation gap.
	**How does gradient boosting differ from a random forest?**	A forest averages deep trees grown side by side on bootstrap samples. Boosting adds shallow trees in a sequence, each fitting the residual errors so far.
	**Logistic regression scores 0.755 and a forest 0.748. Which do you ship?**	The logistic regression: the scores are close, so choose the simpler model (parsimony), and say what would change your mind.

Save your notebook with the comparison table and the decision memo

What you keep from today

Keep your model comparison table: train, cross-validated and test AUC for four models on the Nile Microfinance loans, plus a three-sentence model decision memo that says which model you would ship and why.

Our question today

How do we get a stronger model without fooling ourselves?

Next session

Session 18: Finding structure without labels. We group customers with k-means and flag unusual orders.

Take away

Your lab file

Comparison table

The decision

Memo: which model

Next session

Groups, no labels